# Designing Multiple Queues to Support Asynchronous Traffic Shaping in TSN Switches

Wenxue Wu, Tong Zhang ✉, Zhen Li, Liwei Zhang, Hao Yang, and Fengyuan Ren ✉, *Senior Member, IEEE*

***Abstract*—Asynchronous Traffic Shaping (ATS) extends conventional synchronized Time-Sensitive Networking (TSN) with an asynchronous scheduler to guarantee bounded end-to-end delay. Specifically, ATS introduces additional shaping queues prior to standard priority queues, eliminating the need for global clock synchronization and specific periodic traffic patterns. However, the current queue structure in TSN switches, inherited from Ethernet, does not support ATS. In this paper, we leverage the potential of virtual queues in the shared buffer switch to design multiple queues (MQ) that support ATS. The proposed design organizes queues as logical entities, with all queues sharing a common buffer space, achieving high resource utilization. We prototype MQ with ATS functionality on FPGA switches and evaluate its performance on hardware platforms. Experimental results show that our design introduces negligible additional end-to-end delay, maintains line-rate throughput, and incurs minimal resource overhead while satisfying the timing and shaping requirements defined by ATS.**

***Index Terms*—Time-Sensitive Networking, Multiple Queues, Asynchronous Traffic Shaping, Shared Buffer Switch**

## I. Introduction

The integration of Information Technology (IT) and Operational Technology (OT) is a key enabler of Industry 4.0, breaking down information silos, improving production efficiency, and reducing costs [1]. However, IT and OT have different requirements: IT demands high bandwidth, while OT prioritizes real-time communication and reliability. To bridge this gap, Time-Sensitive Networking (TSN) [2], a set of Ethernet enhancements, has been introduced. Central to TSN are traffic scheduling and shaping mechanisms. For example, the Time-Aware Shaper (TAS) [3] periodically executes preconfigured Gate Control Lists (GCLs) to regulate Ethernet's eight priority queues, thereby ensuring deterministic end-to-end delay. Cyclic Queuing and Forwarding (CQF) [4] forwards frames in fixed cycles by alternating enqueue and dequeue operations between two queues.

However, gate operations in TAS and CQF must adhere to fixed time intervals, which restricts Scheduled Traffic (ST) to strictly periodic flows [5]. Transmitting within preconfigured time slots not only reduces bandwidth efficiency [6] but also demands highly precise clock synchronization. Even minor deviations in synchronization accuracy can result in scheduling failures [7]. To address these limitations and provide real-time guarantees without relying on global synchronization, the IEEE 802.1Qcr standard [8] introduced Asynchronous Traffic Shaping (ATS), which has since been integrated into the TSN standards. ATS guarantees bounded end-to-end delay for time-critical flows while imposing no restrictions on traffic patterns, whether periodic or sporadic. This flexibility reduces deployment complexity and hardware cost, making ATS well suited for large-scale or dynamic networks where precise clock synchronization is impractical. ATS achieves this using the two-level queuing structure shown in Fig. 1, where shaping queues for each input port and priority (forming a *group*) are placed ahead of the standard priority queues. Before entering a shaping queue, ATS computes the frame eligibility time (`FET`) using a token-bucket algorithm. A frame becomes eligible for transmission only when the local time reaches its `FET`. However, the large number of shaping queues required by ATS introduces significant challenges for hardware deployment.

Several studies have evaluated ATS through simulation and theoretical analysis [9]–[13]. However, hardware implementations of ATS remain scarce. Most commercial TSN switches adhere to the traditional Ethernet model with eight fixed priority queues and thus lack the additional shaping queues required by ATS [14]–[17]. Recent work, such as OQ-FIFO [18], has implemented ATS on output queue (OQ) switch architectures by instantiating FIFO IP cores to serve as both memory and queues [5]. Despite implementing basic ATS functionality, these designs suffer from significant on-chip Static Random-Access Memory (SRAM) consumption: each queue stores complete frame payloads in dedicated storage, and buffer space cannot be shared across queues, leading to poor memory efficiency. In contrast, most modern TSN switches adopt the shared buffer architecture [17], [19]–[21], where all frame payloads reside in a centralized on-chip memory and queues are managed via lightweight metadata structures. By decoupling metadata management from payload storage, this architecture can support a significantly larger number of shaping queues without expanding the physical buffer, making it a promising platform for ATS deployment.

In this paper, we propose a scalable multiple queues (MQ) architecture that enables efficient hardware support for ATS in shared buffer TSN switches. Unlike previous designs that utilize dedicated FIFOs for each shaping queue, MQ implements each queue as a lightweight logical entity managed by a "plane", comprising *head* and *tail* pointers along with a

✉ Corresponding authors: Tong Zhang, Fengyuan Ren;

Wenxue Wu, Zhen Li, Liwei Zhang, Hao Yang are with the School of Information Science and Engineering, Lanzhou University, Lanzhou 730000, China (email: wuwx2024@lzu.edu.cn, zhenli2024@lzu.edu.cn, 120220909241@lzu.edu.cn, yanghao21@lzu.edu.cn).

Tong Zhang is with the College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics, Nanjing 210016, China (e-mail: zhangt@nuaa.edu.cn).

Fengyuan Ren is with the School of Information Science and Engineering, Lanzhou University, Lanzhou 730000, China, and also with the Department of Computer Science and Technology, Tsinghua University, Beijing 100084, China (e-mail: renfy@tsinghua.edu.cn).

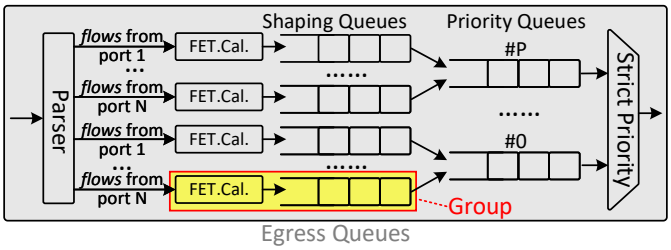



Fig. 1: IEEE 802.1Qcr queue structure

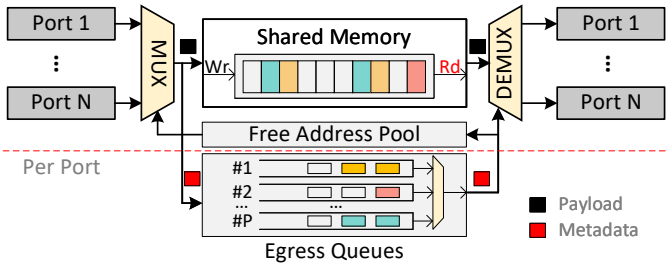



Fig. 2: Typical shared buffer switch architecture

shift register for FETs. All frame payloads are stored once in a centralized shared memory, fully conforming to the one-pipeline switching architecture (1PSA) used in commercial switches. To support additional queues, MQ only requires adding more planes, each composed of registers.

We have implemented MQ on a Xilinx ZYNQ-7000 FPGA. Experimental results indicate that MQ adds only 200 ns of processing delay for 64-Byte frames compared to a baseline without ATS, while maintaining line-rate throughput for all frame sizes. Supporting 96 shaping queues requires merely 28 BRAMs, which stands in sharp contrast to the 384 BRAMs consumed by the state-of-the-art design. Memory usage remains constant as the number of queues increases, while logic resources and registers grow only marginally. Across varied traffic patterns, MQ precisely enforces rate and burst constraints, achieving full compliance with IEEE 802.1Qcr. To the best of our knowledge, this is the first FPGA-based implementation of a resource-efficient, highly scalable multiple queues architecture that supports ATS.

The rest of this paper is organized as follows. Section II presents the background. Section III discusses the motivation. Section IV describes the detailed design of MQ. Section V covers the hardware implementation. Section VI evaluates the performance of MQ. Section VII reviews related work. Finally, Section VIII concludes the paper.

## II. Background

This section first reviews the traffic shaping mechanisms in TSN, then examines the IEEE 802.1Qcr queue architecture, and finally outlines the frame queuing architectures adopted in contemporary switches.

### A. Traffic Shaping Mechanisms in TSN

TSN switches have traditionally relied on synchronous traffic shaping and scheduling mechanisms, such as TAS and CQF, which depend on global time synchronization and allocate fixed transmission slots. Specifically, TAS employs an offline GCL to reserve exclusive transmission windows for time-critical flows, thereby isolating them from best-effort traffic. CQF alternates enqueue and dequeue operations across two queues, such that a frame received in slot $t$ is transmitted in slot $t+1$. Both mechanisms operate strictly in periodic, cycle-based transmission windows. Because transmission opportunities are available only at predetermined cycles, frames arriving outside their assigned windows must wait until the next cycle, introducing additional end-to-end delay. Consequently, TAS and CQF are ill-suited for non-periodic traffic such as sporadic, bursty, or event-driven flows [5], [7], despite their effectiveness for periodic time-critical flows. This limits their applicability in modern industrial networks, which typically combine diverse traffic types under rate constraints.

To address the limitations of synchronous mechanisms, asynchronous mechanisms have been integrated into the TSN standards. Among these, ATS provides per-flow, hop-by-hop shaping tailored to individual flow profiles. Unlike the Credit-Based Shaper (CBS) [22], which regulates traffic at the priority level, ATS enables fine-grained, flow-level regulation through a token bucket algorithm at each egress port. ATS maintains a per-flow credit counter that increments at Committed Information Rate (CIR) up to Committed Burst Size (CBS). The Frame Eligibility Time (FET) is computed as the earliest time at which a frame becomes eligible for transmission, determined by the current credit level and the CIR. This token-bucket-based computation allows frames to become eligible as soon as sufficient credit is accumulated, enabling ATS to naturally adapt to both periodic and non-periodic traffic patterns without relying on fixed transmission cycles.

However, this flexibility comes at a cost. Ideal ATS requires a shaping queue for each flow, often far more than the limited queue resources available in commercial TSN switches.

### B. IEEE 802.1Qcr Queue Structure

In the standardized ATS mechanism defined in IEEE 802.1Qcr [8], a compromise is struck between per-flow and per-class granularity. ATS employs a *per-group* model, where flows originating from the same ingress port and sharing the same priority are aggregated into a single *group*, as illustrated in Fig. 1. Based on the configured Committed Information Rate (CIR) and Committed Burst Size (CBS), the Shaper Eligibility Time (SET) is calculated as the earliest possible transmission time for a frame, determined by the current Bucket Empty Time (BET) and the CIR.

A fundamental property of the standardized ATS is that all flows in a group share a Group Eligibility Time (GET). This GET is updated to the Frame Eligibility Time (FET) of the most recently transmitted frame in that group. Let FAT denote the frame arrival time. A frame's FET is determined as $\mathtt{FET} = \max(\mathtt{FAT}, \mathtt{GET}, \mathtt{SET})$, as shown in Algorithm 1, ensuring that the sequence of FETs in the same group is non-decreasing and that transmission ordering is preserved [8]. ATS also defines a Maximum Residence Time (MRT) that limits how long a

**Algorithm 1** Frame Eligibility Time Calculation Algorithm

$T_{EmptyToFull} = CBS/CIR$
$SET = BET + frame.length/CIR$
$T_{BucketFull} = BET + T_{EmptyToFull}$
$FET = \max(FAT, GET, SET)$
**if** $FET \leq FAT + MRT$ **then**
    $GET = FET$
    **if** $FET < T_{BucketFull}$ **then**
        $BET = SET$
    **else**
        $BET = SET + FET - T_{BucketFull}$
    **end if**
    ASSIGNANDPROCESS($frame$, $FET$)
**else**
    DISCARD($frame$)
**end if**

frame may remain in a switch. A frame is discarded if its FET exceeds its FAT plus the MRT [23].

IEEE 802.1Qcr realizes ATS through a two-level queuing architecture in which shaping queues are placed ahead of the traditional priority queues at each egress port, as shown in Fig. 1. Under this structure, each *group* is assigned one shaping queue, and the total number of shaping queues at an egress port scales with $(N-1) \times P$, where $N$ is the number of ports and $P$ is the number of priorities. For example, a four-port switch with eight priorities requires 24 shaping queues at each egress port, significantly exceeding the typical 8 queues provided by commercial TSN switches.

### C. Frame Queuing Architecture

Switches commonly employ four principal frame queuing architectures: input queue (IQ), output queue (OQ), combined input and output queue (CIOQ), and combined input and crosspoint queue (CICQ). In these architectures, queues are deployed at the ingress ports, egress ports, or crosspoints of the switch to support buffering and scheduling operations. Each port in these designs is typically equipped with dedicated on-chip memory operating at line rate. For example, OQ-FIFO [18] adopts the OQ switch architecture, in which arriving frames are buffered at the egress ports and all queuing and scheduling decisions are performed on the actual frame payloads. Specifically, this design instantiates dedicated FIFO IP cores at each egress port to serve as hardware queues. However, because these FIFO buffers are statically allocated and cannot be shared across queues, overall buffer utilization is reduced and scheduling flexibility is significantly constrained.

Recent advances in memory technology have enabled SRAM to achieve access speeds well above typical line rates, supporting multi-port access through time-division multiplexing. This progress has facilitated the adoption of shared buffer architectures in OQ switches (e.g., 100 Gbps for 20-port switches [24], 10 Gbps for 64-port switches [25]). Compared to distributed memory structures, shared buffer architectures offer lower implementation complexity and more efficient utilization of on-chip SRAM for time-critical traffic, and thus have been widely adopted in TSN switches [17], [19], [20], [26]. Due to the ample bandwidth of SRAM, shared buffer switches typically adopt the 1PSA, where a common pipeline processes frames from all ports sequentially. At any given moment, only one frame is processed, consistent with the characteristics of dual-port SRAM, where only one read or write operation can be performed per clock cycle, as shown in Fig. 2. As incoming frames are segmented into fixed-size cells and written to shared memory, the corresponding *metadata*, containing the buffer address, is linked to virtual output queues. These virtual queues do not consume physical buffer resources; rather, they exist purely as logical entities.

Although ATS defines multiple shaping queues, as illustrated in Fig. 1, its hardware implementation is inherently constrained by the shared buffer switch architecture shown in Fig. 2. All shaping queues access the shared memory through a single interface using time division multiplexing. Consequently, only one queue can be serviced at any time, and each enqueue or dequeue operation updates its *metadata* while issuing at most one read or one write to the shared memory. Thus, all multiple queue operations must adhere to the physical constraints of shared buffer switch pipelines.

## III. Motivation

ATS is particularly attractive because it can achieve bounded end-to-end delay without relying on global clock synchronization or specific traffic patterns. Despite its theoretical benefits, deploying ATS in practical TSN switches faces significant hardware constraints. Commercial TSN switches typically implement a small and fixed number of egress queues per port, usually eight queues per port associated with IEEE 802.1Q priority classes. These queues are static and cannot be extended. As a result, they are fundamentally incompatible with ATS, which requires additional shaping queues.

Recent efforts to support ATS, such as OQ-FIFO [18], instantiate dedicated FIFO IP cores as shaping queues. Although functionally correct, this approach follows an OQ architecture, in which each shaping queue stores complete frame payloads. This design leads to two key limitations. First, each shaping queue occupies a dedicated physical buffer and therefore cannot exploit the shared buffer architecture adopted by most industrial TSN switches, where frame payloads are stored in centralized memory to maximize buffer utilization [17], [19], [20], [26]. Second, the BRAM consumption of this approach grows rapidly with the number of shaping queues.

In contrast, shared buffer switch architectures perform queuing using *metadata* rather than frame payloads. This allows shaping queues to be implemented as lightweight logical entities, enabling the number of shaping queues to scale without increasing the physical buffer size. For example, implementing 96 shaping queues in OQ-FIFO consumes 384 BRAM blocks, which nearly exhausts the available on-chip SRAM resources. In contrast, MQ supports the same number of queues using only 28 BRAM blocks.

Considering the above-mentioned advantages, we employ MQ, a scalable, metadata-driven logical queuing architecture specifically designed for ATS in shared buffer TSN switches.

## IV. Multiple Queues Design

In this section, we first provide an overview of the proposed multiple queues in the shared buffer switch, followed by a

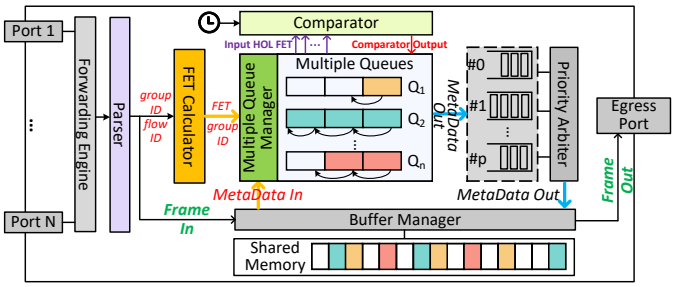



Fig. 3: The organization of MQ in the shared buffer switch (the original switch modules depicted in gray and the MQ represented in color)

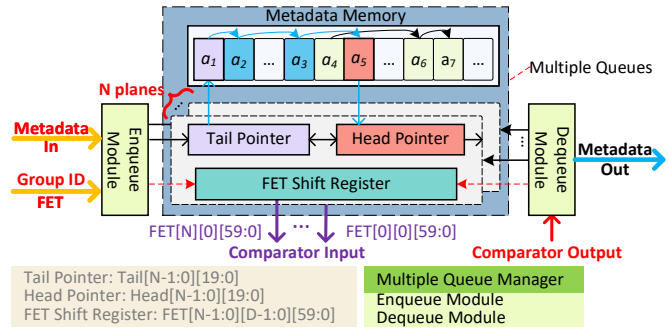



Fig. 4: The structure of MQ planes. It comprises three main arrays: (i) `Tail[N-1:0][19:0]`: $N$ tail pointers, each 20 bits wide; (ii) `Head[N-1:0][19:0]`: $N$ head pointers, each 20 bits wide; (iii) `FET[N-1:0][D-1:0][59:0]`: $N$ FET Shift Registers, each with depth $D$ and 60-bit width.

detailed description of the key modules.

### A. Overview

Fig. 3 presents a high-level overview of the proposed architecture, showing only one egress port for clarity. The processing pipeline begins when frames enter the switch, where the **Forwarding Engine** determines the correct egress port for each frame. The **Parser** then extracts key attributes including the *flow ID*, the *group ID*, and the frame length $L$ to support subsequent processing. The **Buffer Manager** allocates buffer addresses based on the frame size and generates the corresponding *metadata*, which is subsequently forwarded to the **Multiple Queues (MQ)** module. In parallel, the **FET Calculator** computes the `FET` for the frame based on its flow information. Within the **MQ** module, the *metadata* is inserted into the appropriate virtual queue according to the *group ID*. Each virtual queue is represented as an independent linked list that maintains its own logical ordering. The **Comparator** examines the `FET` of the head-of-line frame in each queue and selects the frame that has become eligible for transmission. Eligible frames are forwarded from the MQ module to the priority queues in accordance with ATS scheduling rules. After a frame is dequeued from the priority queues, the corresponding payload is retrieved from the shared memory and transmitted through the egress port.

In the following sections, we describe the design and management of each module in detail.

### B. Multiple Queues

In the shared buffer switch, MQ does not allocate physically independent memory for each queue. Instead, as shown in Fig. 4, the design provides a logical plane for every queue, where each plane maintains its own *head* pointer and *tail* pointer that define the logical boundaries of the queue. The remaining elements of the queue are organized as a linked list and stored centrally in the Metadata Memory.

All queues share a common Metadata Memory implemented as a dual-port SRAM. Although the queues are logically distinct, the behavior of the shared memory ensures that only one queue is serviced in any given cycle. This matches the execution model of the underlying shared buffer architecture.

The Metadata Memory stores the linked list nodes for all queues. The *head* pointer identifies the first entry of a queue, while the *tail* pointer identifies the last. For example, in the top plane of Fig. 4, controlling only the *head* pointer $a_5$ and the *tail* pointer $a_1$ is sufficient to operate the entire queue. This linked list structure allows each queue to grow according to traffic demand without requiring static depth allocation. Creating a new queue only requires allocating a new logical plane and initializing its pointers. To coordinate with the Comparator, each queue maintains an FET Shift Register that stores all `FET`s. The `FET` of the head-of-line frame is forwarded to the Comparator for scheduling decisions.

During the enqueue operation, the Multiple Queues Manager selects the appropriate plane based on the *group ID*, writes the metadata entry into Metadata Memory at the location indicated by the *tail* pointer of that plane, and then updates the *tail* pointer. During the dequeue operation, the Manager reads the metadata entry referenced by the *head* pointer and updates the pointer to the next entry in the linked list.

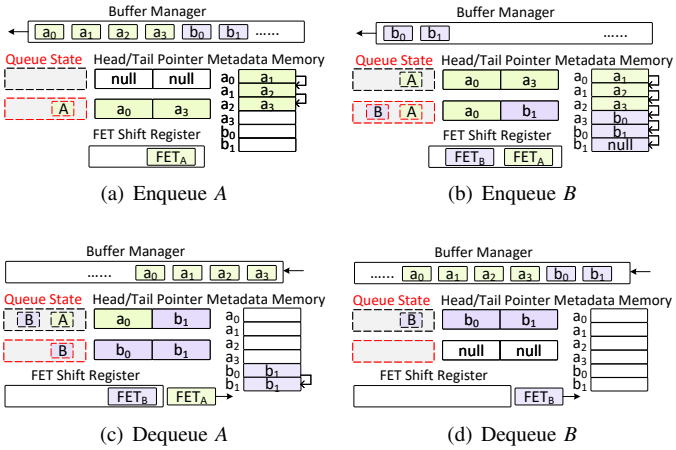
All enqueue and dequeue operations, including pointer management and Metadata Memory access, are coordinated by the centralized Multiple Queues Manager.

### C. Multiple Queues Manager

The Multiple Queues Manager, a centralized hardware module shared by all virtual queues of MQ, is responsible for executing the core queue management operations: assigning incoming frames to the correct queue and dequeuing frames for transmission. It comprises two primary submodules: **Enqueue Module** and **Dequeue Module**, as shown in Fig. 4.

**Enqueue Module.** Enqueue Module processes each incoming frame after its `FET` has been computed by the FET Calculator. Before enqueueing, the frame is checked against its maximum residence constraint. If the computed `FET` exceeds the sum of the `FAT` and the configured `MRT`, the frame is immediately dropped to prevent buffer overflow and unbounded end-to-end delay. If the frame passes the `MRT` check, it is forwarded to the Enqueue Module.

The Enqueue Module first determines the target queue by performing a lookup based on the corresponding *group ID*. It then writes the frame's *metadata* into the Metadata Memory at the location indicated by the *tail* pointer of the selected queue,




(a) Enqueue *A* (b) Enqueue *B*

(c) Dequeue *A* (d) Dequeue *B*

Fig. 5: Example of enqueue and dequeue operations on a selected virtual queue.

thereby logically appending the frame to the end of the queue's linked list. The computed FET is subsequently written to the end of the corresponding FET Shift Register.

This sequence of operations is illustrated in Fig. 5(a) and Fig. 5(b), which depict the states of the *head* and *tail* pointers, the Metadata Memory, and the FET Shift Register before and after enqueueing Frames A and B. Initially, the queue is empty, with both the *head* and *tail* pointers set to *null*. As shown in Fig. 5(a), when frame A arrives, the Buffer Manager allocates buffer addresses $a_0$, $a_1$, $a_2$, and $a_3$ according to the frame size and writes the corresponding payload cells into the shared memory. The Enqueue Module detects that the queue is empty, initializes a new linked list by setting $a_0$ as the *head* pointer, and sequentially writes $a_1$, $a_2$, and $a_3$ into the Metadata Memory. Upon completion, $a_3$ is assigned as the *tail* pointer, and $FET_A$ is appended to the FET Shift Register. As shown in Fig. 5(b), when frame B is enqueued, its buffer addresses are appended starting from the current *tail* pointer $a_3$. After inserting the metadata entries associated with frame B, the *tail* pointer is updated to $b_1$.

**Dequeue Module.** A dequeue operation is initiated when the Comparator asserts a dequeue signal, indicating that the head-of-queue frame in a given queue has become eligible for transmission. Upon receiving this signal, the Dequeue Module identifies the corresponding target queue by *group ID*.

The module first removes the FET of the head-of-queue frame from the FET Shift Register and shifts the remaining entries forward by one position. The updated FET of the new head-of-queue frame is then presented to the Comparator for subsequent eligibility evaluation. Meanwhile, the module uses the *head* pointer of the selected queue to fetch the corresponding frame metadata from the Metadata Memory. Once the metadata is retrieved, the *head* pointer advances to the next linked-list entry, thereby logically dequeuing the frame. The extracted metadata is finally forwarded to the priority queues for final scheduling and transmission.

This process is shown in Fig. 5(c) and Fig. 5(d), which depict the state transitions during the dequeue operation on a selected queue. When frame A is dequeued, its $FET_A$ entry is removed from the FET Shift Register, and the *metadata* entries $(a_0, a_1, a_2, a_3)$ associated with frame A are sequentially popped from the Metadata Memory. The *head* pointer is then updated to $b_0$, which marks the beginning of the next frame in the queue. The dequeue operation for frame B follows the same procedure, as shown in Fig. 5(d). After the *metadata* entries of frame B are removed, both the *head* and *tail* pointers become *null*, indicating that the queue is now empty.

The design of the Multiple Queues Manager ensures that queue management is decoupled from the physical buffering of payloads. It operates solely on *metadata*, *head/tail* pointers, and FET values, simplifying control logic.

### D. Discussion

**Queue Size and Scalability.** In MQ, all queues are virtual and their sizes are not statically bounded. Queue sizes can grow naturally with traffic load because the Multiple Queues Manager manages only *metadata*, such as *head* and *tail* pointers, while all payloads remain in the shared memory.

MQ's scalability stems from its support for an arbitrary number of shaping queues, each corresponding to a unique *group* (combination of an ingress port and a priority class at a given egress port). Rather than assigning dedicated FIFOs for payload storage, MQ represents every shaping queue as a lightweight logical structure composed only of a *head* and *tail* pointer pair. Adding new queues thus requires only additional pointer pairs without extra physical FIFOs or memory. Consequently, the total number of shaping queues scales linearly with the number of traffic classes and ports, while the physical buffer footprint remains unchanged. This property makes MQ well suited for future TSN switches with high port density and finer-grained traffic classification.

**Worst-Case Execution Time.** To ensure that the MQ satisfies TSN's stringent end-to-end delay guarantees, we analyze the worst-case execution time (WCET) for queue operations. In this design, each frame payload is segmented into fixed-size cells and stored in shared memory, while the corresponding *metadata* entries are written to a dedicated Metadata Memory. Both memory modules are implemented using independent SRAM, enabling payload and *metadata* to be accessed in parallel in the same clock cycle. This parallelism eliminates memory access contention and ensures high throughput for queue operations.

Let $L$ denote the frame length, let $C = 64$ Bytes denote the cell size, and let $n = \lceil L/C \rceil$ denote the number of cells per frame. Define $t_{\text{ptr}}$ as the access latency for the queue pointers; $t_{\text{mw}}$ and $t_{\text{mr}}$ as the latencies for writing and reading a *metadata* entry, respectively; and $t_{\text{cw}}$ and $t_{\text{cr}}$ as the latencies for writing and reading a cell.

*a) Enqueue:* Each enqueue operation consists of two pointer accesses (*tail* pointer read and update), and $n$ concurrent writes to the Metadata Memory and shared memory. The worst-case enqueue latency is:

$$T_{\text{enq}}(L) = 2t_{ptr} + n \cdot \max(t_{mw}, t_{cw}) \tag{1}$$

*b) Dequeue:* Dequeue operations follow the same process as enqueue (*head* pointer read and update), except that reads replace writes:

$$T_{\text{deq}}(L) = 2t_{ptr} + n \cdot \max(t_{mr}, t_{cr}) \tag{2}$$

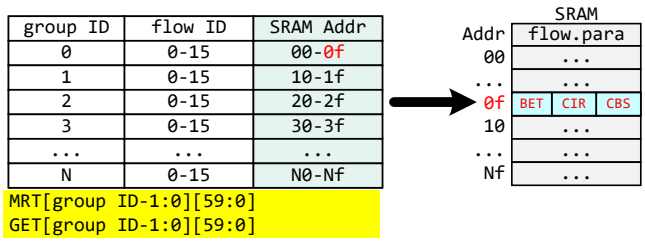



Fig. 6: Flow and group parameter configuration

On the target FPGA platform with dual-port SRAMs, we have $t_{ptr} = t_{mw} = t_{mr} = t_{cw} = t_{cr} = 3$ clock cycles. For a 1500-Byte frame, $n = \lceil 1500/64 \rceil = 24$, thus:

$$T_{\text{enq}} = T_{\text{deq}} = (2 + 24) \times 3 = 78 \text{ cycles} \tag{3}$$

This corresponds to approximately 624 ns at a 125 MHz clock frequency. As shown in Table I, this queuing latency constitutes only a small fraction of the total switch processing delay. Moreover, because dequeue from the MQ module is fully overlapped with enqueue into the priority queues, the effective per-frame scheduling overhead is just 78 cycles, which is orders of magnitude lower than in conventional OQ-FIFO designs [18].

## V. Implementation

This section outlines the key implementation considerations and details the core supporting components required to enable MQ in a shared buffer switch.

### A. Implementation Considerations

Modern network devices are increasingly equipped with programmable hardware to support evolving protocols. Many contemporary TSN switches, such as the Xperis OpenBox-S4 [27] (based on Xilinx ZYNQ series), Kontron's PCIE-0400-TSN NIC [28] (featuring an Altera Cyclone V FPGA), and the AMD 100M/1G TSN Subsystem [29], offer hardware programmability, enabling fine-grained control over switching behavior and protocol implementation.

The modular architecture and well-defined timing semantics of the MQ enable straightforward integration into commercial TSN platforms with minimal engineering effort. In this work, we develop a prototype TSN switch based on the Xilinx ZYNQ-7000 System on Chip (SoC) platform [30]. All functional modules are implemented from scratch in Verilog HDL and fully synthesized for deployment on the FPGA fabric.

The MQ hardware implementation is publicly available at https://github.com/dhf1999sd/MQ.

### B. Supporting Components

*1) FET Shift Register:* To efficiently store and update the `FET` for each enqueued frame, we implement the *FET Shift Register* as a shift register structure. This design inherently preserves FIFO order: each `FET` entry remains temporally aligned with its corresponding frame in the queue at all times. When the head-of-queue frame is dequeued, its associated `FET` is immediately removed from the head of the register, and all subsequent `FETs` are shifted forward by one position.

The shift operation is realized through parallel data-path propagation in the FPGA, completing in one clock cycle and thus demonstrating constant-time ($\mathcal{O}(1)$) performance.

*2) Flow and Group Parameter Configuration:* As previously described, each ATS group is associated with a shaping queue that aggregates multiple flows within the same group. In our implementation, each group supports up to 16 individual flows. To optimize memory usage and access efficiency, we employ a shared SRAM to store the shaping parameters for all groups, as shown in Fig. 6.

Each flow is uniquely identified by a combination of its *group ID* and *flow ID* (0 to 15), which together form the address for SRAM access. The data stored at each address consists of the shaping parameters for the corresponding flow, including the `CIR`, `CBS`, and `BET`.

In addition, per-group parameters such as `MRT` and `GET` are maintained in dedicated register arrays. These registers are accessed and updated during the shaping process to guarantee proper temporal ordering of frames.

## VI. Evaluation

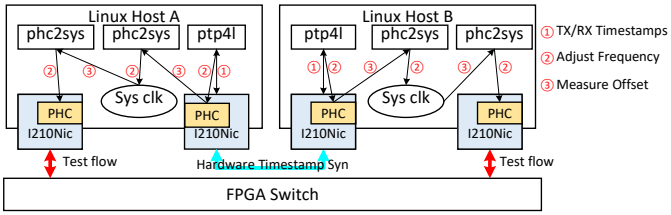
This section aims to answer the following three questions:

1) **Basic Performance:** Does MQ introduce only negligible additional end-to-end delay while sustaining line-rate throughput?
2) **Resource Overhead:** How does the resource utilization of MQ compare with conventional OQ-FIFO designs, and can MQ scale to support a larger number of shaping queues?
3) **ATS Functionality:** Can MQ seamlessly integrate with the `FET` calculation mechanism and accurately control rate and burst size constraints in accordance with ATS specifications under diverse traffic patterns?

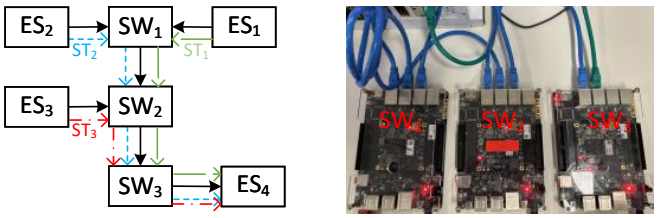
To address these questions, we adopt a bottom-up evaluation methodology. We first conduct single-node microbenchmarks to assess end-to-end delay and throughput, followed by an in-depth analysis of component-level latency performance and resource utilization. Next, we verify the functional correctness of the ATS. Finally, we evaluate system-level performance under a realistic multi-node industrial network scenario.

### A. Single-node Microbenchmark

*1) Experiment Setup:* **Topology and Settings:** The microbenchmark testbed consists of a single FPGA switch and two Linux-based test hosts, as illustrated in Fig. 7(a). Each host is equipped with an Intel Core i5-14600K CPU, 32 GB of RAM, and two Intel I210 network interface cards (NICs) that support nanosecond-level hardware timestamping via the Precision Time Protocol (PTP) hardware clock (PHC). To avoid interference from unrelated traffic and ensure accurate end-to-end delay measurements, the test traffic is configured as shown in Fig. 7(a). On each host, one NIC is dedicated exclusively to time synchronization using `ptp4l` from the LinuxPTP suite [31], while the other NIC is used solely for traffic generation or reception. Two instances of `phc2sys` are

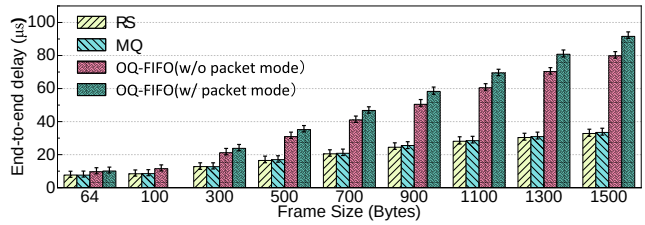



(a) Single-node microbenchmark

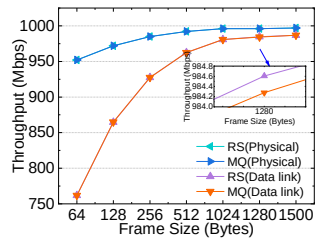



(b) Multi-node topology

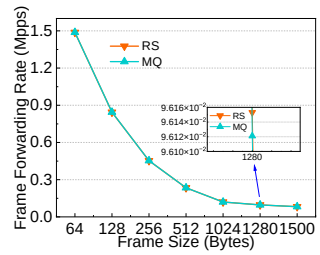
Fig. 7: Experiment setup




(a) End-to-end delay




(b) Throughput




(c) Frame forwarding rate

Fig. 8: Basic transmission performance

deployed on each host to synchronize the PHC and system clocks, ensuring that transmit and receive timestamps are aligned within 100 ns.

**Measurement and Metrics:** End-to-end delay is measured as the time difference between frame transmission and reception using NIC-generated hardware timestamps with sub-microsecond accuracy, which are collected at the receiver via `tcpdump` with hardware timestamping enabled. The residual time synchronization error introduced by PTP is on the order of tens of nanoseconds and is therefore negligible compared to the microsecond-level end-to-end delays reported in this work. Jitter is computed as the standard deviation of per-frame end-to-end delays. Throughput and frame forwarding rate are measured at the receiver. FPGA resource consumption is obtained from post-implementation reports generated by the Vivado Design Suite 2023.2 [32].

**Traffic Generation:** For end-to-end delay measurements, we generate time-critical traffic using Ziggo-TSNperf [33], which enables precise frame transmission based on NIC hardware timestamps. For throughput evaluation, full-load traffic is generated at 1 Gbps using Pktgen-DPDK [34]. The ATS parameters are configured with a `CIR` of 1000 Mbps and a `CBS` of 1500 Bytes to avoid rate limiting. This setup isolates the impact of ATS and allows us to assess the intrinsic forwarding capability of MQ.

**Baselines:** For a fair and consistent comparison, we consider the following three queuing architectures:

- **RS:** A reference TSN switch that supports eight priority queues per port, following the conventional Ethernet priority model. The design is implemented on the AMD Xilinx ZYNQ-7000 platform [30].
- **MQ:** The proposed MQ architecture in this work, synthesized and evaluated on the same ZYNQ-7000 platform.
- **OQ-FIFO:** An OQ switch architecture that instantiates dedicated FIFO IP cores for each queue to support ATS. We reproduce the design proposed in [18] on the ZYNQ-7000 platform and evaluate its end-to-end delay under both packet-mode-enabled and packet-mode-disabled configurations. When packet mode is enabled, each FIFO buffers the entire frame payload before forwarding, which introduces additional processing and queuing latency; when packet mode is disabled, full-frame buffering is avoided, resulting in lower latency. Resource utilization results for OQ-FIFO are cited directly from [18].

*2) Experiment Results:* **End-to-end Delay:** For each experiment, we collect 10000 frames at the receiver and measure the end-to-end delay of each frame. The results are shown in Fig. 8(a). As expected, the end-to-end delay increases with frame size. Importantly, the additional end-to-end delay introduced by MQ is negligible compared to the RS and remains substantially lower than that of OQ-FIFO [18], regardless of whether packet mode is enabled. Moreover, the performance advantage of MQ becomes increasingly evident as the frame size grows. For instance, when the frame size is 1500 Bytes, MQ achieves an average end-to-end delay of 33.448 $\mu$s, whereas OQ-FIFO exhibits an average delay of 79.819 $\mu$s with packet mode disabled and 91.562 $\mu$s with packet mode enabled.

This behavior is primarily attributed to the design of OQ-FIFO, which relies on hardware FIFOs to buffer the complete frame payload. As the frame size increases, multiple read and write cycles are required across several FIFO-based modules, causing the processing latency to grow proportionally with frame size. In contrast, MQ stores frame payloads once in a centralized shared memory and performs queue management using lightweight metadata. As a result, each frame incurs only minimal memory access overhead, leading to significantly lower end-to-end delay.

**Throughput:** The throughput results, shown in Fig. 8(b) and Fig. 8(c), demonstrate that MQ consistently achieves line-rate performance (1 Gbps) across all evaluated frame sizes. Similar to RS, MQ sustains stable throughput at both the physical layer and the data-link layer, indicating that it does

TABLE I: Component-Level Latency Comparison

| Queuing Architecture | Payload (B) | Rx-MAC | Forwarding Engine | Parser | FET Calculator | Multiple Queues | Priority Queues | Tx-MAC | Total (ns) |
|---|---|---|---|---|---|---|---|---|---|
| **RS** | 64 | 766 | 104 | 32 | – | – | 120 | 832 | 1854 |
| | 100 | 1046 | 112 | 56 | – | – | 224 | 1128 | 2566 |
| | 300 | 2646 | 312 | 48 | – | – | 536 | 2824 | 6366 |
| | 500 | 4246 | 512 | 40 | – | – | 848 | 4528 | 10174 |
| | 700 | 5846 | 712 | 32 | – | – | 1160 | 6232 | 13982 |
| | 900 | 7446 | 912 | 32 | – | – | 1472 | 7944 | 17806 |
| | 1100 | 9046 | 1112 | 80 | – | – | 1888 | 9008 | 21134 |
| | 1300 | 10646 | 1312 | 72 | – | – | 2200 | 9120 | 23350 |
| | 1500 | 12246 | 1512 | 64 | – | – | 2512 | 9216 | 25550 |
| **MQ** | 64 | 766 | 104 | 56 | 152 | 112 | 32 | 832 | 2054 |
| | 100 | 1062 | 128 | 80 | 152 | 176 | 48 | 1160 | 2806 |
| | 300 | 2662 | 328 | 72 | 152 | 368 | 160 | 2944 | 6686 |
| | 500 | 4262 | 528 | 64 | 161 | 551 | 256 | 4744 | 10566 |
| | 700 | 5862 | 728 | 56 | 152 | 752 | 264 | 6632 | 14446 |
| | 900 | 7456 | 934 | 56 | 152 | 944 | 336 | 8456 | 18334 |
| | 1100 | 9062 | 1128 | 104 | 152 | 1200 | 432 | 9688 | 21766 |
| | 1300 | 10662 | 1328 | 96 | 152 | 1392 | 504 | 9912 | 24046 |
| | 1500 | 12262 | 1528 | 88 | 152 | 1584 | 768 | 9944 | 26326 |
| **OQ-FIFO [18]** | **Payload (B)** | **Eth_Ctrl** | **Pre-Sw FIFO** | **Set A.T.** | **Calc. E.T.** | **FDB Switch** | **ATS Queue+ E.T. Gate** | **FIFO** | **Total (ns)** |
| | 64 | 1248 | 584 | 8 | 1392 | 792 | 1296 | 24 | 5906 |
| | 100 | 1824 | 872 | 8 | 1968 | 1088 | 1872 | 24 | 8210 |
| | 300 | 4984 | 2472 | 8 | 5152 | 2784 | 5344 | 24 | 21010 |
| | 500 | 8184 | 4072 | 8 | 8352 | 4384 | 8544 | 24 | 33810 |
| | 700 | 11384 | 5672 | 8 | 11552 | 5888 | 11744 | 24 | 46610 |
| | 900 | 14584 | 7272 | 8 | 14752 | 7488 | 14944 | 24 | 59410 |
| | 1100 | 17784 | 8872 | 8 | 17952 | 9088 | 18144 | 24 | 72210 |
| | 1300 | 20984 | 10472 | 8 | 21152 | 10688 | 21344 | 24 | 85010 |
| | 1500 | 24184 | 12072 | 8 | 24352 | 12288 | 24544 | 24 | 97810 |

not introduce throughput bottlenecks in the switch processing pipeline. The measured frame forwarding rates in Fig. 8(c) further support this observation.

For example, when the frame size is 1280 Bytes, the measured physical layer throughput is identical for MQ and RS at 996 Mbps. At the data-link layer, MQ achieves a throughput of 984.27904 Mbps, which is only 0.03328% lower than RS. Similarly, the measured frame forwarding rate is 0.09612 Mpps, representing a marginal difference of only 0.03120% compared to RS.

**Pipeline Latency:** To precisely quantify the pipeline latency introduced by MQ, we use the Xilinx Integrated Logic Analyzer (ILA) [35] to measure the cycle consumption of each pipeline component. Each experiment is repeated 5 times, and no variation in the measured latency is observed across runs. As expected, the number of consumed cycles depends on the frame size. As summarized in Table I, MQ introduces only a minimal increase in pipeline latency compared to the RS, adding approximately 200 ns for 64-Byte frames. In contrast, OQ-FIFO [18] incurs a significantly higher latency overhead, which becomes more pronounced as the frame size increases. For example, when processing a 1500-Byte frame, OQ-FIFO requires 97.8 $\mu$s, whereas MQ consumes only 26.326 $\mu$s.

Similar to the end-to-end delay results, this difference arises because OQ-FIFO relies on multiple FIFO-based modules whose processing latency scales proportionally with the frame size, as these FIFOs temporarily buffer the entire frame payload. In contrast, MQ stores frame payloads only once in a shared memory and performs queue management using lightweight metadata, thereby achieving substantially lower pipeline latency.

**Resource Usage:** The implemented switch supports 512 MAC entries, a 32.7 KBytes shared buffer, 8 priority levels, and 4 ports. For a direct and fair comparison with OQ-FIFO [18], all measurements are conducted on the Xilinx Kintex-7 KC705 evaluation platform. A total of 96 queues are instantiated in hardware, matching the configuration used in OQ-FIFO. The overall FPGA resource utilization is summarized in Table II. The results show that the Multiple Queues module accounts for only 2.70%, 8.50%, and 13.26% of the total BRAM, LUT, and FF resources, respectively, indicating a modest overall hardware footprint. To ensure a fair comparison, we further isolate and compare functionally equivalent modules in both designs, highlighted in blue in Table II. Specifically, we compare the *Multiple Queues*, *Priority Queues*, and *Comparator* modules in MQ with the corresponding *Trans.Sel.* module in OQ-FIFO. The results show that MQ consumes only 28 BRAM blocks (6.22%), whereas OQ-FIFO requires 384 BRAM blocks (86.3%). This substantial reduction stems from MQ's virtual queue design, which decouples queue management from payload storage.

To evaluate the scalability of MQ, we further synthesize the core modules, including the *Multiple Queues*, *Priority Queues*, and *Comparator*, with varying numbers of queues per port, ranging from 24 to 128. The results are summarized in Table III. As the number of queues increases, BRAM

TABLE II: FPGA Resource Consumption

| Config | Module | BRAM (%) | LUT (%) | FF (%) |
|---|---|---|---|---|
| MQ | MAC (4) | 12.0(2.68) | 3190(1.56) | 5787(1.41) |
| | Parser | 0.0(0.0) | 256(0.13) | 316(0.08) |
| | FET Calculator (4) | 8(1.80) | 3928(1.92) | 8612(2.12) |
| | Buffer Manager | 0.5(0.11) | 77(0.04) | 91(0.02) |
| | Multiple Queues (4) | 12(2.70) | 17321(8.50) | 54064(13.26) |
| | Priority Queues (4) | 16(3.52) | 928(0.32) | 1984(0.64) |
| | Comparator (4) | 0(0) | 13,052(6.61) | 9,717(2.26) |
| | Shared Memory | 7.5(1.69) | 0(0.0) | 0(0.0) |
| | Others | 20.5(4.61) | 824(0.40) | 3097(0.76) |
| | **Total** | **76.5(17.19)** | **39576(19.42)** | **83668(20.53)** |
| OQ-FIFO [18] | MAC | 8.0(1.8) | 9832(4.8) | 15379(3.8) |
| | Pre-switch FIFO | 0.0(0.0) | 5477(2.7) | 3664(0.9) |
| | Scheduler | 16.0(3.6) | 65879(32.3) | 50456(12.4) |
| | FDB Switch | 2.0(0.5) | 17539(8.6) | 15015(3.7) |
| | Trans. Sel. | 384.0(86.3) | 51736(25.4) | 41296(10.1) |
| | Others | 3.0(0.7) | 5097(2.5) | 7396(1.8) |
| | **Total** | **413.0(92.8)** | **155827(76.5)** | **133627(32.8)** |

TABLE III: Resource utilization for different numbers of queues

| Number of Queues | BRAM (%) | LUT (%) | FF (%) |
|---|---|---|---|
| 24 | 7 (1.57%) | 7816 (3.84%) | 16466 (4.04%) |
| 56 | 7 (1.57%) | 15910 (7.81%) | 35587 (8.73%) |
| 72 | 7 (1.57%) | 20198 (9.91%) | 45201 (11.09%) |
| 96 | 7 (1.57%) | 26006 (12.76%) | 59221 (14.53%) |
| 128 | 7 (1.57%) | 29839 (14.64%) | 72711 (17.84%) |

utilization remains constant at 7 blocks. In contrast, LUT and FF utilization grow linearly with the number of queues. This trend is expected, as each additional queue introduces only a small amount of per-queue state, including *head* and *tail* pointers and FET Shift Registers, implemented using registers.

### B. ATS Function Verification

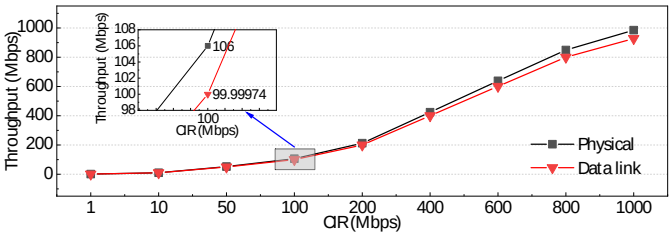
We evaluate the ability of the MQ to support ATS functionality from two key perspectives: (i) rate control effectiveness, and (ii) burst size enforcement capability.

*1) Experiment Setup:* The experimental testbed and measurement methodology follow those described in Section VI-A. Traffic is generated using Pktgen-DPDK.

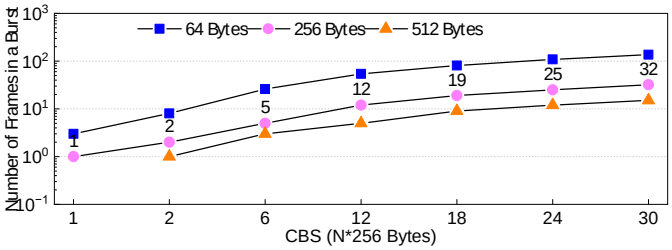
To validate rate control behavior, the transmission rate is configured at 1 Gbps with the `CBS` fixed at 1500 Bytes, while the `CIR` is gradually varied from 1 Mbps to 1000 Mbps. Under this configuration, ATS is expected to discard excess frames and regulate the output throughput to conform strictly to the configured `CIR`.

To evaluate burst handling behavior, the sender transmits frames at line rate (1 Gbps). Under this condition, the theoretical inter-frame interval depends on the frame size. For example, when the frame size is 256 Bytes, the theoretical inter-frame interval is 2.208 µs[1]. Similarly, the inter-frame interval is 672 ns for 64-Byte frames and 4.256 µs for 512-Byte frames. The `CIR` is set to 100 Mbps. Under this configuration, the theoretical inter-frame interval is 22.08 µs when the frame size is 256 Bytes[2]. Correspondingly, the theoretical minimum inter-frame interval is 6.72 µs for 64-Byte frames and 42.56 µs

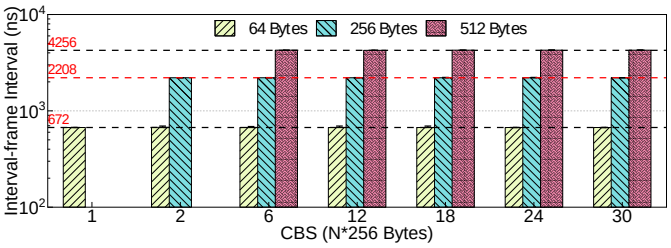
[1]This value is calculated as $\frac{(256+12+8)\times 8}{1000\times 10^6} = 2.208\ \mu s$, where the additional 12 and 8 bytes account for the inter-frame gap and the preamble/SFD.
[2]Calculated as $\frac{(256+12+8)\times 8}{100\times 10^6} = 22.08\ \mu s$.



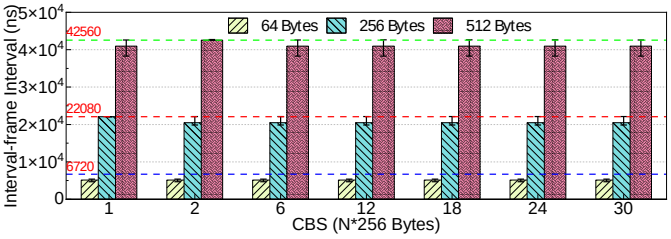

(a) Rate control under varying CIR




(b) Number of frames within a burst




(c) Inter-frame interval within the burst




(d) Inter-frame interval after burst completion

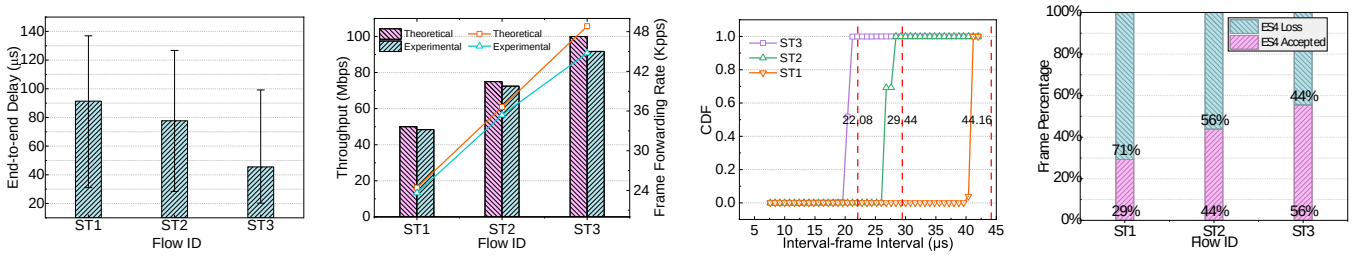
Fig. 9: Transmission behavior under different CIR and CBS.

for 512-Byte frames. The `CBS` is varied from $1 \times 256$ Bytes to $30 \times 256$ Bytes to evaluate the ability of ATS to enforce different burst constraints.

At the receiver side, `tcpdump` with hardware timestamping enabled is used to precisely record inter-frame intervals.

*2) Experiment Results:* **Rate Control:** As shown in Fig. 9(a), the measured Layer 2 bandwidth at the receiver closely follows the configured `CIR` over the entire tested range, confirming that ATS implemented with MQ accurately enforces rate constraints.

**Burst Size Control:** For each experimental configuration, 100000 frames were collected at the receiver. Fig. 9(b) summarizes the observed burst behavior under different `CBS` settings. The measured burst length increases consistently with the configured `CBS`. For instance, with a `CIR` of 100 Mbps and a `CBS` of $18 \times 256$ Bytes, the measured number of frames in a burst is 19 when the frame size is 256 Bytes. The same trend




(a) End-to-end delay (b) Throughput & frame forwarding rate (c) Inter-frame intervals (d) Frame drop rate

Fig. 10: Multi-node transmission performance

is consistently observed across all tested configurations.

The burst length also scales as expected with the frame size. When the frame size increases to 512 Bytes, the number of frames per burst decreases to 9, approximately half of that observed with 256-Byte frames. Conversely, when the frame size is reduced to 64 Bytes, the burst length increases to 81 frames, which is consistent with the higher packetization rate under the same `CBS` constraint.

The measured inter-frame interval within a burst for 256-Byte frames is 2.208 µs, which exactly matches the sender-side inter-frame interval, as indicated by the red line in Fig. 9(c). Although frames are sent in bursts, the inter-frame interval within each burst is still constrained by the physical link, because frames are serialized on the wire and cannot arrive with a zero or near-zero gap. After the burst is exhausted, the mean inter-frame interval increases to approximately 22.08 µs, consistent with the theoretical interval determined by the configured `CIR`, as indicated by the red line in Fig. 9(d).

These results demonstrate that MQ enforces CBS-based burst constraints while preserving CIR-based rate control, in accordance with the ATS specification.

### C. Multi-node Performance

*1) Experiment Setup:* To further evaluate the effectiveness of the MQ in supporting ATS, we conducted a multi-node experiment based on an industrial automation network topology described in IEC/IEEE 60802 [36]. The testbed comprises three TSN switches (SW1, SW2, SW3) and four end stations (ES1, ES2, ES3, ES4), as shown in Fig. 7(b).

**Traffic Configuration:** Three ST flows are assigned the same highest priority and originate from $\mathrm{ES}_1$, $\mathrm{ES}_2$, and $\mathrm{ES}_3$, with $\mathrm{ES}_4$ serving as the common destination. Each ST flow transmits traffic at a rate of 165 Mbps using a frame size of 256 Bytes. Flows $\mathrm{ST}_1$ and $\mathrm{ST}_2$ generate periodic traffic, while $\mathrm{ST}_3$ produces traffic with random inter-frame intervals.

At the switches, bandwidth constraints are enforced by configuring the `CIR` to 50 Mbps, 75 Mbps, and 100 Mbps for $\mathrm{ST}_1$, $\mathrm{ST}_2$, and $\mathrm{ST}_3$, respectively, with the `CBS` fixed at 256 Bytes. Traffic transmission and reception are performed using a four-port Intel i350 NIC, which also provides hardware timestamping support for accurate timing measurements.

**ATS Group Setting:** Each ATS group corresponds to a dedicated queue. In $\mathrm{SW}_1$, $\mathrm{ST}_1$ and $\mathrm{ST}_2$ are assigned to distinct ATS groups, resulting in separate queues. In $\mathrm{SW}_2$, $\mathrm{ST}_1$ and $\mathrm{ST}_2$ share one ATS group, while $\mathrm{ST}_3$ occupies an independent group with its own queue. In $\mathrm{SW}_3$, all ST flows are aggregated into a single ATS group, sharing one unified queue.

**Metrics and Theoretical Baselines:** We evaluate MQ using the following metrics: end-to-end delay, throughput, frame forwarding rate, inter-frame interval, and frame drop rate. End-to-end delay is measured by synchronizing the clocks of the four NIC ports using the `phc2sys` tool and computing the difference between the hardware transmit timestamp at the sender and the hardware receive timestamp at the receiver. Throughput and frame forwarding rate are obtained by counting the total received Bytes and frames at the receiver. Inter-frame intervals are calculated as the time difference between consecutive frames of the same flow observed at the receiver. The frame drop rate is computed as the ratio between the number of frames dropped and the number of frames transmitted at the sender. A total of 1046386 frames were collected over a 10 s measurement period.

The theoretical baseline values are summarized as follows:

- **Throughput:** 50 Mbps for $\mathrm{ST}_1$, 75 Mbps for $\mathrm{ST}_2$, and 100 Mbps for $\mathrm{ST}_3$.
- **Frame forwarding rate:** 24.414 kpps for $\mathrm{ST}_1$, 36.621 kpps for $\mathrm{ST}_2$, and 48.828 kpps for $\mathrm{ST}_3$.
- **Inter-frame interval:** 44.16 µs for $\mathrm{ST}_1$, 29.44 µs for $\mathrm{ST}_2$, and 22.08 µs for $\mathrm{ST}_3$.
- **Frame drop rate:** 69.70% for $\mathrm{ST}_1$, 54.55% for $\mathrm{ST}_2$, and 39.39% for $\mathrm{ST}_3$.

*2) Experiment Results:* **End-to-end delay:** As shown in Fig. 10(a), all ST flows achieve bounded end-to-end delays. The average measured end-to-end delays are 91.34 $\mu$s (maximum 136.96 $\mu$s) for $\mathrm{ST}_1$, 77.69 $\mu$s (maximum 126.74 $\mu$s) for $\mathrm{ST}_2$, and 45.53 $\mu$s (maximum 99.175 $\mu$s) for $\mathrm{ST}_3$. These results demonstrate that ATS provides microsecond-level end-to-end delay guarantees; however, it cannot effectively suppress end-to-end delay jitter.

**Throughput and frame forwarding rate:** Fig. 10(b) presents throughput and frame forwarding rate using two vertical axes: the left y-axis shows throughput as a bar chart, and the right y-axis shows frame forwarding rate as a line plot. The results show that the admitted bandwidth of each flow remains close to its configured `CIR`, with measured values of 48.35 Mbps for ST1, 72.48 Mbps for ST2, and 91.72 Mbps for ST3. Correspondingly, the measured frame forwarding rates are 23.61 kpps, 35.39 kpps, and 44.78 kpps for ST1, ST2, and

ST3, respectively, remaining close to their theoretical values.

**Inter-frame intervals:** Analysis of Fig. 10(c) shows that the measured mean inter-frame intervals of $\mathrm{ST}_1$, $\mathrm{ST}_2$, and $\mathrm{ST}_3$ are $41.25$ $\mu$s, $27.44$ $\mu$s, and $22.12$ $\mu$s, respectively, remaining close to their corresponding theoretical on-wire values of $44.16$ $\mu$s, $29.44$ $\mu$s, and $22.08$ $\mu$s. All relative deviations are below $7\%$[3]. These results indicate that the ATS implementation can effectively regulate inter-frame intervals according to the configured `CIR`. Nevertheless, measurable inter-flow interference still exists under concurrent transmission.

**Frame drop rate:** As depicted in Fig. 10(d), the measured frame drop rates remain close to the theoretical values. For $\mathrm{ST}_1$, the measured drop rate is $71\%$, which is slightly higher than the theoretical value but still follows the expected trend determined by the difference between the sender rate and the configured `CIR`. Similar behavior is observed for $\mathrm{ST}_2$ and $\mathrm{ST}_3$, whose measured drop rates also remain close to their corresponding theoretical values.

Overall, the experimental results demonstrate that the proposed MQ mechanism adequately satisfies the requirements for ATS deployment.

## VII. Related Work

**Asynchronous Traffic Shaping in TSN:** ATS has emerged as a key mechanism for enabling deterministic communication in industrial and automotive networks. Originally proposed as the Urgency-Based Scheduler (UBS) in 2016 [37], ATS was subsequently standardized as part of TSN in IEEE 802.1Qcr [8]. The foundational UBS work introduced analytical models for the Length-Rate Quotient (LRQ) and Token Bucket Emulation (TBE) algorithms, and derived end-to-end delay bounds. These derived bounds were further validated through simulations.

Building on the theoretical foundations of ATS, prior work has extensively investigated its performance characteristics, parameter sensitivity, and scheduling behavior. The majority of existing studies have primarily relied on theoretical analysis or simulation-based evaluations [10]–[13]. For instance, Mohammadpour *et al.* [12] provide a comprehensive performance comparison between ATS and CBS. Similarly, other works [10], [38] evaluate TAS and ATS using the OMNeT++ simulator to determine whether ATS can achieve performance comparable to that of TAS. Zhou *et al.* [39] further examine ATS behavior and highlight a fundamental trade-off: while TAS offers strong traffic isolation, it requires strict global time synchronization; in contrast, ATS eliminates this requirement, thereby significantly reducing system complexity and deployment costs. Fang *et al.* [40] demonstrate that ATS performance is highly dependent on the selection of the `CIR` and `CBS`: overly conservative settings may result in worse performance than CBS or strict-priority scheduling. Hu *et al.* [41] investigate the impact of `CBS` on end-to-end delay bounds. Additionally, other works [42]–[45] explore priority assignment and scheduling optimization for shaping queues of ATS, employing techniques such as SMT-based optimization, deep reinforcement learning, and greedy algorithms to meet individual flow deadlines.

Existing hardware implementations of ATS remain limited. For example, Zhou *et al.* [5] and Akram *et al.* [18] realize ATS by instantiating dedicated hardware queues using FIFO IP cores. While these designs are functionally correct and closely follow the ATS specification, they adopt an OQ architecture in which each shaping queue directly buffers complete frame payloads. Such designs are inherently incompatible with the shared buffer architecture adopted by most industrial TSN switches, where payloads are stored in a centralized memory to maximize buffer utilization.

**Multiple Queues in TSN:** The concept of multiple queues has been explored in TSN to improve schedulability and bandwidth utilization [46]–[48]. SBQ [48], for example, separates short and long non-time-critical frames into distinct FIFO queues. By classifying frames according to their lengths, TAS gate control parameters can be tailored to each queue. In particular, queues buffering short frames can be assigned smaller guard bands, thereby reducing bandwidth waste caused by overly conservative guard intervals. Bujosa *et al.* [47] exploit multiple time-critical queues to resolve ordering conflicts in multi-hop scheduling. Compared to constraint-based scheduling with a single queue, their approach significantly improves schedulability while guaranteeing zero reception jitter. HPFS [46] introduces per-flow virtual queues via dynamic hashing to enhance TSN schedulability and reduce end-to-end delay. Beyond academic proposals, the IETF DetNet working group has proposed a three-queue CQF mechanism (CQF-3) [49], which introduces an additional queue to buffer late time-critical frames and transmit them within the current cycle, thereby preserving the ping-pong forwarding semantics. Similarly, the Paternoster scheduler [50] guarantees deterministic latency for time-critical traffic without requiring global time synchronization by allocating four cyclic queues per traffic class at each port.

From a hardware perspective, supporting a large number of queues remains challenging. Nikologiannis *et al.* [51] propose a hierarchical memory architecture that stores payloads in off-chip DRAM and metadata in on-chip SRAM, enabling a large number of queues at the cost of increased latency and system complexity. OQ-FIFO [18] implements multiple queues by replicating FIFO IP cores for each shaping queue. While straightforward to implement, this approach suffers from poor scalability, as the required on-chip memory resources grow linearly with the number of queues and ports, making it unsuitable for resource-constrained TSN switches.

In contrast to prior work, MQ provides a standard-compliant ATS implementation while remaining fully compatible with shared buffer switch architectures. By realizing multiple shaping queues as metadata-driven logical structures rather than dedicated payload buffers, MQ achieves substantially lower hardware overhead and enables support for ATS in resource-constrained TSN switches.

## VIII. Conclusion

In this paper, we present MQ, a novel multiple-queue architecture that enables practical hardware support for ATS

[3]Relative errors are calculated as $|I_{\mathrm{obs}} - I_{\mathrm{theo}}|/I_{\mathrm{theo}}$, yielding $6.59\%$ for $\mathrm{ST}_1$, $6.79\%$ for $\mathrm{ST}_2$, and $0.18\%$ for $\mathrm{ST}_3$.

in shared buffer TSN switches. By representing each shaping queue as a lightweight logical plane composed of head and tail pointers and an FET Shift Register, MQ decouples queue management from physical payload storage while remaining fully compatible with shared buffer switch architectures. MQ is tightly integrated into the TSN switch processing pipeline, with all enqueue and dequeue operations synchronized to the shared memory interface. We implement MQ on a Xilinx ZYNQ-7000 FPGA and evaluate it through comprehensive experiments, including single-node microbenchmarks, ATS functional verification, and multi-node performance tests. Experimental results demonstrate that MQ sustains line-rate throughput, introduces only 200 ns of additional end-to-end delay for 64-Byte frames, and accurately enforces ATS rate and burst constraints. Notably, MQ supports the same number of shaping queues using only 28 BRAM blocks, representing an order-of-magnitude reduction in memory consumption compared to state-of-the-art designs. Overall, MQ provides a practical and resource-efficient foundation for deploying ATS in real-world TSN switches, bridging the gap between ATS theory and scalable hardware implementation.

## IX. Acknowledgments

This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant Nos. 62132007, 62472219, and 62221003, as well as gifts from Huawei and Broadcom.

The authors acknowledge the use of ChatGPT as a linguistic tool to improve the presentation of the initial draft. All technical content, experiments, analyses, figures, and conclusions were independently developed and validated by the authors.

## References

[1] E. Kuppusamy and K. Mariappan, "Integration of operational technology (OT) and information technology (IT) through intelligent automation in manufacturing industries," in *Advances in Manufacturing Technology XXXIV*. IOS Press, 2021, pp. 284–290.

[2] IEEE, "Time-sensitive networking (TSN) task group," [Online]. Available: https://1.ieee802.org/tsn/, accessed: 30 August 2025.

[3] "IEEE standard for local and metropolitan area networks—bridges and bridged networks—amendment 25: Enhancements for scheduled traffic," *IEEE Std 802.1Qbv-2015 (Amendment to IEEE Std 802.1Q-2014 as amended by IEEE Std 802.1Qca-2015, IEEE Std 802.1Qcd-2015, and IEEE Std 802.1Q-2014/Cor 1-2015)*, pp. 1–57, 2016.

[4] "IEEE standard for local and metropolitan area networks—bridges and bridged networks—amendment 29: Cyclic queuing and forwarding," *IEEE 802.1Qch-2017 (Amendment to IEEE Std 802.1Q-2014 as amended by IEEE Std 802.1Qca-2015, IEEE Std 802.1Qcd-2015, IEEE Std 802.1Q-2014/Cor 1-2015, IEEE Std 802.1Qbv-2015, IEEE Std 802.1Qbu-2016, IEEE Std 802.1Qbz-2016, and IEEE Std 802.1Qci-2017)*, pp. 1–30, 2017.

[5] Z. Zhou, M. S. Berger, and Y. Yan, "Mapping TSN traffic scheduling and shaping to FPGA-based architecture," *IEEE Access*, vol. 8, pp. 221 503–221 512, 2020.

[6] A. Nasrallah, A. S. Thyagaturu, Z. Alharbi, C. Wang, X. Shao, M. Reisslein, and H. ElBakoury, "Ultra-low latency (ULL) networks: The IEEE TSN and IETF DetNet standards and related 5G ULL research," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 1, pp. 88–145, 2019.

[7] H. Yang, T. Zhang, X. Feng, Y. Ma, L. Zhang, and F. Ren, "Absorbing time synchronization errors using shadow queues in time-sensitive networking," *IEEE Internet of Things Journal*, vol. 12, no. 9, pp. 12 912–12 926, 2025.

[8] "IEEE standard for local and metropolitan area networks—bridges and bridged networks—amendment 34: Asynchronous traffic shaping," *IEEE Std 802.1Qcr-2020 (Amendment to IEEE Std 802.1Q-2018 as amended by IEEE Std 802.1Qcp-2018, IEEE Std 802.1Qcc-2018, IEEE Std 802.1Qcy-2019, and IEEE Std 802.1Qcx-2020)*, pp. 1–151, 2020.

[9] T. Lübeck, P. Meyer, T. Häckel, F. Korf, and T. C. Schmidt, "Asynchronous traffic shaping and redundancy: Avoiding unbounded latencies in in-car networks," in *Proc. IEEE Vehicular Networking Conf. (VNC)*, 2025, pp. 1–8.

[10] A. Nasrallah, A. S. Thyagaturu, Z. Alharbi, C.-X. Wang, X. Shao, M. Reisslein, and H. Elbakoury, "Performance comparison of IEEE 802.1 TSN time-aware shaper (TAS) and asynchronous traffic shaper (ATS)," *IEEE Access*, vol. 7, pp. 44 165–44 181, 2019.

[11] A. Grigorjew, F. Metzger, T. Hossfeld, and J. Specht, "A simulation of asynchronous traffic shapers in switched ethernet networks," in *Proc. International Conf. on Networked Systems (NetSys)*, Mar. 2019.

[12] E. Mohammadpour, E. Stai, M. Mohiuddin, and J.-Y. L. Boudec, "Latency and backlog bounds in time-sensitive networking with credit based shapers and asynchronous traffic shaping," in *Proc. 30th International Teletraffic Congress (ITC 30)*, 2018, pp. 1–6.

[13] L. Zhao, P. Pop, and S. Steinhorst, "Quantitative performance comparison of various traffic shapers in time-sensitive networking," *IEEE Transactions on Network and Service Management*, vol. 19, no. 3, pp. 2899–2928, 2022.

[14] Kyland, "SICOM3000TSN," [Online]. Available: https://www.kyland.com.cn/products/SICOM3000TSN.html, accessed: 30 August 2025.

[15] CAST, "Tsn-sw tsn ethernet switch product brief," [Online]. Available: https://www.cast-inc.com/sites/default/files/pdfs/2022-10/cast_tsn-sw-xilinx.pdf, accessed: 30 August 2025.

[16] Planet, "TSN-5225-4T," [Online]. Available: https://planet.com.br/produtos/tsn-5225-4t, accessed: 30 August 2025.

[17] Microchip, "SparX-5i L2/L3 TSN-enabled industrial ethernet switch," [Online]. Available: https://www.microchip.com/en-us/product/vsc7558tsn, accessed: 30 August 2025.

[18] A. B. Ahmed, T. Hirofuchi, and T. Fukai, "Hardware design and evaluation of an fpga-based network switch supporting asynchronous traffic shaping for time sensitive networking," *IEEE Access*, vol. 12, pp. 123 149–123 165, 2024.

[19] J. Yan, W. Quan, X. Yang, W. Fu, Y. Jiang, H. Yang, and Z. Sun, "TSN-builder: Enabling rapid customization of resource-efficient switches for time-sensitive networking," in *Proc. ACM/IEEE Design Automation Conf. (DAC'20)*, 2020, pp. 1–6.

[20] Z. Li, H. Wan, Y. Deng, X. Zhao, Y. Gao, X. Song, and M. Gu, "Time-triggered switch-memory-switch architecture for time-sensitive networking switches," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 1, pp. 185–198, 2020.

[21] W. Quan, W. Fu, J. Yan, and Z. Sun, "OpenTSN: An open-source project for time-sensitive networking system development," *CCF Transactions on Networking*, vol. 3, no. 1, pp. 51–65, 2020.

[22] "IEEE standard for local and metropolitan area networks—bridges and bridged networks," *IEEE Std 802.1Q-2022 (Revision of IEEE Std 802.1Q-2018)*, pp. 1–2163, 2022.

[23] M. Boyer, "Equivalence between the urgency based shaper and asynchronous traffic shaping in time-sensitive networking," *Leibniz Transactions on Embedded Systems*, vol. 9, no. 1, pp. 1:1–1:27, 2024.

[24] Broadcom, "Trident3-X5/BCM56770 series," [Online]. Available: https://www.broadcom.com/products/ethernet-connectivity/switching/strataxgs/bcm56770-series, accessed: 30 August 2025.

[25] S. Das and R. Sankar, "Broadcom smart-buffer technology in data center switches for cost-effective performance scaling of cloud applications," [Online]. Available: https://www.broadcom.com/collateral/etp/SBT-ETP100.pdf, accessed: 30 August 2025.

[26] Z. Dai and J. Zhu, "Saturating the transceiver bandwidth: Switch fabric design on FPGAs," in *Proc. ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, 2012, pp. 67–76.

[27] Xperis, "OpenBox-S4 (TSN version)," [Online]. Available: http://www.xperis.com.cn/product.html, accessed: 1 August 2025.

[28] Kontron, "PCIe-0400-TSN network interface card," [Online]. Available: https://www.kontron.com/en/products/pcie-0400-tsn-network-interface-card/p151637, accessed: 1 August 2025.

[29] AMD, "100M/1G TSN subsystem," [Online]. Available: https://www.amd.com/en/products/adaptive-socs-and-fpgas/intellectual-property/1gtsn.html, accessed: 1 December 2025.

[30] Xilinx, "SoCs with hardware and software programmability," [Online]. Available: https://www.amd.com/en/products/adaptive-socs-and-fpgas/soc/zynq-7000.html, accessed: 30 August 2025.

[31] "LinuxPTP," [Online]. Available: https://linuxptp.sourceforge.net/, accessed: 8 April 2025.

[32] AMD, "AMD vivado design suite," [Online]. Available: https://www.amd.com/en/products/software/adaptive-socs-and-fpgas/vivado.html, accessed: 30 August 2025.

[33] "Ziggo-TSNPerf," [Online]. Available: https://github.com/MobiSense/Ziggo-TSNPerf, accessed: 3 December 2025.

[34] K. Wiles, "Pktgen-DPDK," [Online]. Available: https://github.com/pktgen/Pktgen-DPDK, accessed: 8 April 2025.

[35] Xilinx, "Integrated logic analyzer (ILA)," [Online]. Available: https://www.amd.com/en/products/adaptive-socs-and-fpgas/intellectual-property/ila.html, accessed: 30 August 2025.

[36] "Use cases iec/ieee 60802 v1.3," [Online]. Available: https://www.ieee802.org/1/files/public/docs2018/60802-industrial-use-cases-0918-v13.pdf, Sep. 2018, group contribution to IEC/IEEE 60802.

[37] J. Specht and S. Samii, "Urgency-based scheduler for time-sensitive switched ethernet networks," in *Proc. Euromicro Conf. on Real-Time Systems (ECRTS)*, 2016, pp. 75–85.

[38] Ž. Maletić, M. Ljubojević, and M. Savić, "Comparative analysis of time-sensitive network scheduling and traffic shaping mechanisms for A380 aircraft network topology," in *Proc. Telecommunications Forum (TELFOR)*, 2023, pp. 1–4.

[39] Z. Zhou, M. S. Berger, S. R. Ruepp, and Y. Yan, "Insight into the IEEE 802.1 Qcr asynchronous traffic shaping in time-sensitive networking," *Advances in Science, Technology and Engineering Systems Journal*, vol. 4, no. 1, pp. 292–301, 2019.

[40] B. Fang, Q. Li, Z. Gong, and H. Xiong, "Simulative assessments of credit-based shaping and asynchronous traffic shaping in time-sensitive networking," in *Proc. International Conf. on Advanced Infocomm Technology (ICAIT)*, 2020, pp. 111–118.

[41] H. Hu, Q. Li, H. Xiong, and B. Fang, "The delay bound analysis based on network calculus for asynchronous traffic shaping under parameter inconsistency," in *Proc. IEEE International Conf. on Communication Technology (ICCT)*, 2020, pp. 908–915.

[42] J. Specht and S. Samii, "Synthesis of queue and priority assignment for asynchronous traffic shaping in switched ethernet," in *Proc. IEEE Real-Time Systems Symp. (RTSS)*, 2017, pp. 178–187.

[43] L. Zhang, G. Sun, R. Liu, W. Quan, H. Yu, and D. Niyato, "Priority-dominated traffic scheduling enabled ATS in time-sensitive networking," *IEEE Transactions on Network and Service Management*, vol. 22, no. 1, pp. 470–484, 2025.

[44] J. Prados-Garzon, T. Taleb, and M. Bagaa, "LEARNET: Reinforcement learning based flow scheduling for asynchronous deterministic networks," in *Proc. IEEE International Conf. on Communications (ICC)*, Jun. 2020.

[45] J. Prados-Garzon and T. Taleb, "Asynchronous time-sensitive networking for 5G backhauling," *IEEE Network*, pp. 144–151, Mar. 2021.

[46] H. Jing and T. Zhang, "Improving traffic scheduling based on per-flow virtual queues in time-sensitive networking," in *Proc. IEEE Symposium on Computers and Communications (ISCC)*, 2023, pp. 1185–1188.

[47] D. Bujosa, M. Ashjaei, A. V. Papadopoulos, T. Nolte, and J. Proenza, "HERMES: Heuristic multi-queue scheduler for TSN time-triggered traffic with zero reception jitter capabilities," in *Proc. International Conf. on Real-Time Networks and Systems*, 2022, pp. 70–80.

[48] F. Heilmann and G. Fohler, "Size-based queuing: An approach to improve bandwidth utilization in TSN networks," *SIGBED Review*, vol. 16, no. 1, pp. 9–14, Feb. 2019.

[49] N. Finn, J.-Y. L. Boudec, E. Mohammadpour, J. Zhang, B. Varga, and J. Farkas, "Detnet bounded latency," [Online]. Available: https://datatracker.ietf.org/doc/html/draft-finn-detnet-bounded-latency-04, Jun. 2019, internet-Draft.

[50] M. Seaman, "Paternoster policing and scheduling," [Online]. Available: https://www.ieee802.org/1/files/public/docs2019/cr-seaman-paternoster-policing-scheduling-0519-v04.pdf, May 2019, iEEE 802.1 contribution.

[51] A. Nikologiannis, I. Papaefstathiou, G. Kornaros, and C. Kachris, "An FPGA-based queue management system for high-speed networking devices," *Microprocessors and Microsystems*, vol. 28, no. 5, pp. 223–236, 2004, special Issue on FPGAs: Applications and Designs.

**Wenxue Wu** received the B.E. degree in electronic information engineering from Northwestern Polytechnical University, Xi'an, China, in 2021, and the M.S. degree in communication engineering from Lanzhou University, Lanzhou, China, in 2024. He is currently pursuing the Ph.D. degree with the School of Information Science and Engineering, Lanzhou University. His research interests include switch and queue architectures, time-sensitive networking, and FPGA-based system design.

**Tong Zhang** received the B.E. degree in computer science and technology from Xi'an Jiaotong University, Xi'an, China, in 2014, and the Ph.D. degree in computer science and technology from Tsinghua University, Beijing, China, in 2019. She is currently an Associate Professor with the College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics, Nanjing, China. Her research interests include flow scheduling in data center networks and traffic management in time-sensitive networking.

**Zhen Li** received the B.E. degree in Electronic Information Science and Technology from Yanshan University, Qinhuangdao, China, in 2020. He is currently pursuing the Ph.D. degree in Computer Application Technology at Lanzhou University, Lanzhou, China. His research interests include time-sensitive networking, programmable scheduling, and programmable data planes.

**Liwei Zhang** received the M.S. degree in computer application technology from Northeastern University, China, in 2021. He is currently pursuing the Ph.D. degree with Lanzhou University, China. His research interests include reliability and traffic scheduling in time-sensitive networking.

**Hao Yang** received the M.S. degree in software engineering from Lanzhou University of Technology, Lanzhou, China, in 2021. He is currently pursuing the Ph.D. degree with the School of Information Science and Engineering, Lanzhou University, Lanzhou, China. His research interests include time-sensitive networking and the industrial Internet.

**Fengyuan Ren** (Senior Member, IEEE) received the B.A. and M.Sc. degrees in automatic control, and the Ph.D. degree in computer science from Northwestern Polytechnical University, Xi'an, China, in 1993, 1996, and 1999, respectively. He is currently a Professor with the Department of Computer Science and Technology, Tsinghua University, Beijing, China. He was previously affiliated with the Department of Electronic Engineering as a Postdoctoral Researcher from 2000 to 2001 and joined the Department of Computer Science and Technology in 2002. He has authored or coauthored over 80 papers in international journals and conferences. His research interests include network traffic management and control, control in and over computer networks, wireless networks, and wireless sensor networks. He has served as a Technical Program Committee Member and Local Arrangement Chair for various IEEE and ACM international conferences.